

python知识点汇总

python知识点汇总

1.基础知识

- 1.1 变量和类型
- 1.2 变量命名和关键字
- 1.3 输入和输出
- 1.4 类型转换
- 1.5 运算符
- 1.6 `eval()` 函数用法总结
- 1.7 `and` 与 `or` 逻辑运算符总结
- 1.8 `range()`函数
- 1.9 `enumerate`的使用

2.选择和循环语句

- 2.1 选择语句
 - 2.1.1 `if`条件嵌套
 - 2.1.2 `if-elif-else`
- 2.2 循环语句
 - 2.2.1 `while`循环
 - 2.2.2 `for`循环
 - 2.2.3 `break`和`continue`

3.容器类型

- 3.1 元组
- 3.2 列表
- 3.3 字典
- 3.4 字符串
- 3.5 集合
- 3.6 `list`、`tuple`、`set`之间类型转换

4.函数

- 4.1 函数的使用
 - 4.1.1 函数的定义
 - 4.1.2 函数的调用
- 4.2 无参函数
- 4.3 有参函数
 - 4.3.1 参数不定
 - 方法2：包裹位置传递
 - 包裹关键字传递
- 4.4 返回值
- 4.5 匿名函数/`lambda`表达式

5.文件

- 5.1 文件打开与关闭
 - 5.1.1 文件打开函数`open()`
 - 5.1.2 文件关闭函数`close()`
- 5.2 文件读写函数
 - 5.2.1 读数据
 - `read()`函数
 - `readline()`函数
 - `readlines()`函数
 - 5.2.2. 写数据
 - `write()`函数

6.turtle库

- 6.1 `turtle`库的引入
- 6.2 画笔运动命令
- 6.3 画笔控制命令

6.4 全局控制命令

7.Tkinter库的使用

7.1 基础概念

7.2 常用的tk组件

7.2.1 添加标签 (Label)

7.2.2 添加按钮 (Button)

7.2.3 输入框 (Entry)

7.2.4 单选按钮 (Radiobutton)

7.2.5 复选框 (Checkbutton)

7.2.6 下拉菜单 (OptionMenu)

7.3 布局管理

7.3.1 pack布局

pack中的side

pack中的fill

pack中的padding

7.3.2 grid布局

7.3.3 place布局

7.4 事件绑定

8.面向对象

8.1 概念

8.2 类和对象

8.2.1 类的定义

8.2.2 创建对象和调用方法

8.2.3 添加和使用属性

8.2.4 self的作用

8.3 魔法方法

8.3.1 __init__()方法

8.3.2 __str__()方法

8.3.3 __del__()方法

8.3.4 其他专有方法

8.4 继承

8.4.1 继承概念

8.4.2 单继承和多层继承

8.4.3 重新父类方法super()

8.4.4 多继承

8.4.5 私有和继承

8.4.6 多态

8.5 属性和方法

8.5.1 实例属性、类属性定义

8.5.2 类属性和实例属性的区别

8.5.3 实例属性、类属性注意事项

8.5.4 类方法和静态方法

1. 基础知识

1.1 变量和类型

变量：存储数据的容器

定义：

```
1 变量名 = 值
```

类型：区分变量不同的功能

常用的类型：

```
1 整型: int
2 浮点型: float
3 字符串: str
4 布尔: bool
```

举例：

type(变量名) 获取变量的类型，需要打印后，才能看到

```
1  # 定义变量，无需指定类型，自动推导
2  a = 250
3  # type(变量名) 获取变量的类型，需要打印后，才能看到
4  print(type(a)) # 输出结果为: <class 'int'>
5
6  b = 3.14
7  print(type(b)) # 输出结果为: <class 'float'>
8
9  name = 'mike'
10 print(type(name)) # 输出结果为: <class 'str'>
11
12 is_man = False
13 print(type(is_man)) # 输出结果为: <class 'bool'>
```

1.2 变量命名和关键字

变量命名规则：

- 由 **字母**、**下划线** 和 **数字** 组成
- **不能以数字开头**
- **不能与关键字重名**
- 建议不要和类型同名，如：int、str

注意：Python 中的标识符是区分大小写的

关键字：是Python已经使用的了，所以**不允许**开发者自己定义和关键字相同的名字的标识符

1.3 输入和输出

输入：获取键盘输入用()函数，返回的是字符串类型str

语法：

```
1 字符串变量 = input("提示信息: ")
```

```
1  #示例代码:
2
3  password = input("请输入密码:")
4  print('您刚刚输入的密码是:%s' % password)
```

输出：使用print实现输出

```
1  # 打印提示
2  print('hello world')
3  print('萨瓦迪卡---泰语，你好的意思')
```

格式化输出：按一定的格式输出到控制台

- 在 Python 中可以使用 `print` 函数将信息输出到控制台
- 如果希望输出文字信息的同时，一起输出数据，就需要使用到 **格式化操作符**
- `%` 被称为**格式化操作符**，专门用于处理字符串中的格式
 - 包含 `%` 的字符串，被称为 **格式化字符串**
 - `%` 和不同的 **字符** 连用，**不同类型的数据** 需要使用 **不同的格式化字符**

常用格式化字符	含义
%s	字符串
%d	有符号十进制整数，%06d 表示输出的整数显示位数，不足的地方使用 0 补全
%f	浮点数，%.2f 表示小数点后只显示两位

- 语法格式如下：

```
1  print("格式化字符串" % 变量1)
2  print("格式化字符串" % (变量1, 变量2...))
```

1.4 类型转换

类型转换函数

函数	说明
int(x)	将 x 转换为一个整数
float(x)	将 x 转换到一个浮点数
str(x)	将 x 转换到一个字符串

注意：浮点数形式的字符串无法转换为 int，如 int("1.5") 会报错

```
1 # 类型转换示例代码
2 # int(x) 转换为整型
3 # float(x) 转换为浮点型
4 # str(x) 转换为字符串
5
6 a = 123 # 整型
7 # 将整型的a转换为字符串类型，再给a_str赋值
8 a_str = str(a)
9 print(type(a_str)) # 输出结果: <class 'str'>
10 print("a_str = %s" % a_str) # 输出结果: a_str = 123
11
12 b = "250" # 字符串类型
13 # 将字符串的b转换为整型，再给b_int赋值
14 b_int = int(b)
15 print(type(b_int)) # 输出结果: <class 'int'>
16
17 c = '3.14' # 字符串
18 c_float = float(c)
19 print(type(c_float)) # 输出结果: <class 'float'>
```

1.5 运算符

1.算术运算符

下面以 a=10, b=20 为例进行计算

运算符	描述	实例
+	加	两个对象相加 a + b 输出结果 30
-	减	得到负数或是一个数减去另一个数 a - b 输出结果 -10
*	乘	两个数相乘或是返回一个被重复若干次的字符串 a * b 输出结果 200
/	除	b / a 输出结果 2
//	取整除	返回商的整数部分 9//2 输出结果 4, 9.0//2.0 输出结果 4.0
%	取余	返回除法的余数 b % a 输出结果 0
**	指数	a**b 为10的20次方， 输出结果 100000000000000000000

- **注意：**混合运算时，优先级顺序为： `**` 高于 `*` `/` `%` `//` 高于 `+` `-`，为了避免歧义，建议使用 `()` 来处理运算符优先级。

示例代码：□

```
1 # 算术运算符
2 """
3 / 除
4 // 取整数商
5 % 取余数
6 ** 多次方
```

```
7  """
8  a = 5 / 2
9  print(a)
10
11 a = 5 // 2
12 print(a)
13
14 a = 5 % 2
15 print(a)
16
17 a = 2 ** 3
18 print(a)
```

2. 赋值运算符

运算符	描述	实例
=	赋值运算符	把 = 号右边的结果 赋给 左边的变量，如 num = 1 + 2 * 3，结果num的值为7

示例代码：

```
1  # 赋值运算符
2  a = 2
3  a = a + 3
4  print(a)
```

3. 复合赋值运算符

运算符	描述	实例
+=	加法赋值运算符	c += a 等效于 c = c + a
-=	减法赋值运算符	c -= a 等效于 c = c - a
*=	乘法赋值运算符	c *= a 等效于 c = c * a
/=	除法赋值运算符	c /= a 等效于 c = c / a
%=	取模赋值运算符	c %= a 等效于 c = c % a
**=	幂赋值运算符	c **= a 等效于 c = c ** a
//=	取整除赋值运算符	c //= a 等效于 c = c // a

示例代码：

```
1  # 复合赋值运算符
2  a = 2
3  a += 3 # a = a + 3      a %= 2 ==> a = a % 2
4  print(a)
```

4. 逻辑运算符

运算符	描述	判断
and	与	两个条件都满足，结果才为真
or	或	只要满足一个条件，结果就为真
not	非	结果为True时，它为False，结果为False时，它为True

```
1 # 如果语文成绩 和 数学成绩都大于90，输出优秀，否则输出继续加油
2 a = int(input('请输入语文成绩: '))
3 b = int(input('请输入数学成绩: '))
4
5 if a > 90 and b > 90:
6     print('优秀')
7 else:
8     print('继续加油')
```

1.6 eval() 函数用法总结

基本语法

```
1 eval(expression, globals=None, locals=None)
2 eval(字符串形式的表达式)
```

- `expression`：字符串形式的 Python 表达式（**不能是语句**，如 `import`、`for`、`def` 等）。
- `globals` / `locals`：可选，控制表达式可用的变量范围。

示例代码：

```
1 # **数学表达式计算**
2 print(eval("3 * (2 + 4)")) # 输出 18
3
4 # **动态变量代入**
5 print(eval("x + y", {"x": 1, "y": 2})) # 输出3
6
7 ***调用函数**
8 print(eval("len('abc')")) # 输出3
9
10 ***解析结构体**
11 print(eval("{'a': 1, 'b': 2}")) # 输出{'a': 1, 'b': 2}
12
```

1.7 and 与 or 逻辑运算符总结

运算符	返回值规则	示例	结果
and	返回第一个假值或最后一个真值	0 and 3 and []	0
or	返回第一个真值或最后一个假值	None or 0 or 5	5

示例代码：

```
1 print(3 and 4) # 输出 4
2 print(0 and 4) # 输出 0
3
4 print(0 or 4) # 输出4
5 print(3 or 4) # 输出3
```

1.8 range()函数

语法：

```
1 range(起始值，结束值，步长)
2
3 # 当只有一个值的时候：
4 range(5)          # 得到[0,1,2,3,4]数据列表 表示从0-5，包括0，不包括5
5
6 # 当有两个值的时候
7 range(1, 4)       # 得到[1,2,3]数据列表      表示从1-4，包括1，不包括4
8
9 # 当有3个值的时候
10 range(0,10,2)    # 得到[0,2,4,6,8]数据列表 表示从0-10，每次增加2
```

示例代码：

```
1 for i in range(10):
2     print(i)    # 结果输出 0,1,2,3,4,5,6,7,8,9
3
4 for num in range(0,10,2):
5     print(num) # 结果输出 0,2,4,6,8
6
7 for num in range(1,4):
8     print(num) # 结果输出 1,2,3
```

1.9 enumerate的使用

- 通过 for 配合 enumerate 遍历容器同时获取元素索引位置、元素

格式：

```
1 for 索引,值 in enumerate(列表):
2     #做一些事情
```

示例代码：

```
1 user_list = [{'name': 'mike', 'age': 34, 'tel': '110'},
2               {'name': 'yoyo', 'age': 18, 'tel': '120'}]
3
4 # 遍历列表，同时把索引位置能打印
5 # 普通方法实现
6 # 1. 定义索引位置变量
```



```
7 | i = 0
8 | # 2. for遍历列表, 打印: 索引、元素
9 | for user_dict in user_list:
10 |     print(i, user_dict)
11 |     # 3. 索引位置+1
12 |     i += 1
13 |
14 | print('=='*20)
15 |
16 | # 通过enumerate方法实现
17 | # enumerate(容器变量): 获取到: 元素位置, 元素
18 | for i, user_dict in enumerate(user_list):
19 |     print(i, user_dict)
```

运行结果:

```
1 | 0 {'name': 'mike', 'age': 34, 'tel': '110'}
2 | 1 {'name': 'yoyo', 'age': 18, 'tel': '120'}
3 | =====
4 | 0 {'name': 'mike', 'age': 34, 'tel': '110'}
5 | 1 {'name': 'yoyo', 'age': 18, 'tel': '120'}
```

2.选择和循环语句

2.1 选择语句

选择语句使用 if-else 来进行判断

语法格式：

```
1  if 要判断的条件：
2      条件满足时，要做的事情
3  else:
4      条件不满足时，要做的事情
```

示例代码：

```
1  # 1. 输入用户年龄，input返回值是字符串，类型转换为int
2  age = int(input('输入用户年龄: '))
3
4  # 2. if 判断是否满 18 岁：
5  if age >= 18:
6      # 2.1 如果满 18 岁，打印一句话：允许进网吧嗨皮
7      print('允许进网吧嗨皮')
8  # 3. 否则，未满 18 岁
9  else:
10     # 3.1 打印一句话：提示回家写作业
11     print('回家写作业')
```

2.1.1 if条件嵌套

语法格式：

```
1  if 条件1:
2      条件1成立时执行的代码
3      if 条件2:
4          条件2成立时执行的代码
5      else:
6          条件2不成立时执行的代码
7  else:
8      条件1不成立时执行的代码
```

示例代码：

```
1  a = int(input("请输入一个整数: "))
2  if a > 10:
3      if a < 20:
4          print('a在10到20之间')
5      else:
6          print('a大于20')
7  else:
8      print('a小于等于10')
```

2.1.2 if-elif-else

语法格式：

```
1  if 条件1:
2      条件1成立时执行的代码
3  elif 条件2:
4      条件2成立时执行的代码
5      ...
6  else:
7      上述条件都不满足时的代码
```

示例代码：

```
1  a = int(input('请输入一个整数: '))
2
3  if(a>0 and a < 10):
4      print("a在0-10之间")
5
6  elif(a > 10 and a <20):
7      print("a在10-20之间")
8
9  else:
10     print("a不在0-20之间")
```

2.2 循环语句

循环的作用就是让 **指定的代码** 重复的执行

在python中使用while或for来实现循环。

2.2.1 while循环

while语句格式：

```
1  变量设置初始值
2
3  while 条件(判断 变量是否满足条件):
4      条件满足时，做的事情1
5      ... (省略) ...
6
7      修改变量
```

- 注意：`while` 语句中的缩进部分是一个 **独立的代码块**

示例代码：

```
1  """
2  1. 定义一个条件变量，一般赋值为0
3  2. while 判断条件:
4      2.1 满足条件的代码块
5      2.2 条件变量的改变【非常重要】
6  """
```

```

7
8 # 1. 定义一个条件变量，一般赋值为0
9 i = 0
10
11 # 2. while 判断条件：
12 while i < 10:
13     # 2.1 满足条件的代码块
14     print('跑步第%d圈' % (i+1))
15
16     # 2.2 条件变量的改变【非常重要】
17     i += 1

```

2.2.2 for循环

语句格式：

```

1 for 变量 in 可迭代的对象: # 可迭代对象都有：字符串、元组、列表、字典等
2     循环体代码
3
4 # 用法1
5 for i in 变量: #变量：是字符串、列表、字典等
6     循环体代码
7
8 # 用法2
9 for i in range(n): #n: 循环次数
10     循环体代码

```

```

1 # 用法1举例
2 # 变量family，每次输出一个字母
3 a = 'family'
4 for i in a:
5     print(i)
6
7 #方法2
8 a = 'family'
9 for i in range(len(a)): #len(a): 表示a这个字符串的长度
10     print(a[i])
11     print(a[i], end='*') #横着输出，每个字母后面加*

```

2.2.3 break和continue

break: 跳出整个循环，不执行后续重复代码

continue: 跳出当前循环，会接着执行后续代码

示例代码：

```
1 # break举例
2 for i in range(6):
3     if i == 3:
4         break
5     print(i)
6
7 # continue举例
8 for i in range(6):
9     if i == 3:
10        continue
11    print(i)
```

上面两个代码中会出现两个输出

break会输出

```
1 | 0 1 2
```

continue会输出

```
1 | 0 1 2 4 5
```

3.容器类型

3.1 元组

定义：

- 1 元组用 `()` 定义
- 2 用于存储 一串 数据，元素 之间使用 `,` 分隔
- 3 元组的 索引 从 `0` 开始
- 4 元组不可修改

注意：元组定义好之后**不可修改**

示例代码：

```
1 # 列表：列表变量 = [元素1, 元素2, .....]
2 # 元组：元组变量 = (元素1, 元素2, .....)
3 my_tuple = ('mike', 'yoyo', 'lily')
4 #           0       1       2
5
6 # 取出第0个元素
7 print(my_tuple[0]) # 输出: 'mike'
```

- 元组中 **只包含一个元素** 时，需要 **在元素后面添加逗号**

```
1 # 元组只有一个元素的时候，格式：(元素,)
2 my_tuple1 = (251,)
3 # 打印my_tuple1的类型
4 print(type(my_tuple1))
```

元组常用操作

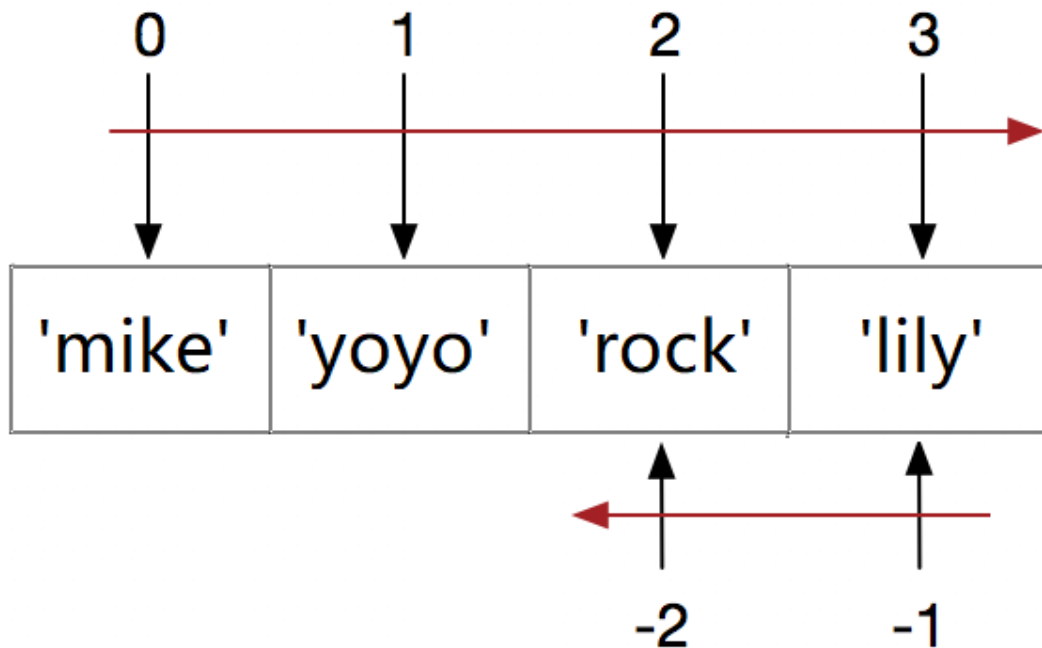
分类	关键字 / 函数 / 方法	说明
查询	元组[索引]	根据索引取值，索引不存在会报错
	len(元组)	元组长度(元素个数)
	if 值 in 元组	判断元组中是否包含某个值

3.2 列表

定义

- 1 列表用 `[]` 定义，元素 之间使用 `,` 分隔
- 2 列表的 索引 从 `0` 开始
- 3 索引 就是元素在 列表 中的位置编号

列表的索引从 0 开始



索引支持负数形式

示例代码：

```
1  """
2  列表定义的格式：
3  列表变量的名字 = [元素1, 元素2, .....]
4
5  使用格式：
6      列表变量[位置]
7      位置：也叫下标、索引
8  """
9
10 # 列表定义
11 name_list = ['mike', 'yoyo', 'rock', 'lily']
12 #           0       1       2       3
13 #          -4       -3       -2       -1
14
15 # 最后一个元素取出来
16 print(name_list[3])
17 print(name_list[-1])
18
19 # 访问列表元素，不要超出范围，不要越界
20 # IndexError: list index out of range
21 # print(name_list[4])
```

- 注意：从列表中取值时，如果 **超出索引范围**，程序会报错

列表常用操作

分类	关键字 / 函数 / 方法	说明
增加	列表.append(值)	在末尾追加数据
	列表.insert(索引, 值)	在索引位置插入数据
	列表.extend([元素1, 元素2, ...,元素n])	向列表末尾增加多个元素
删除	列表.remove(值)	删除第一个出现的指定数据
	列表.pop()	删除最后一个元素
	del 列表[索引]	删除指定索引的数据
修改	列表[索引]='新元素'	修改指定索引的数据，数据不存在会报错
查询	if 值 in 列表	查找元素是否在列表中
	index(值)	查找元素在列表中的哪个位置（返回元素的索引）
	for ... in ...():	循环遍历整个列表
	列表[索引]	根据索引取值，索引不存在会报错
插入	列表(索引, 值)	插入索引位置值，原本位置的值得往后移
	len(列表)	列表长度(元素个数)
	if 值 in 列表:	判断列表中是否包含某个值
排序	列表.sort()	升序排序
	变量 = 列表.sorted()	sorted建立副本后排序
切片	列表[开始索引: 结束索引]	把包含开始，不包含结束的索引值数据输出

相关示例代码

增加操作

```

1  '''增加操作'''
2  foods = ['烧花鸭', '烧雏鸡', '烧子鹅']
3  print('添加前: ', foods)
4
5  # 添加一道新菜“酱鸡” 这样子
6  foods.append('酱鸡')
7  print('添加后: ', foods)
8

```



```

9 # 在"烧花鸭"前面添加"腊肉"这道菜
10 foods.insert(0, '腊肉')
11 print('添加后: ', foods)
12
13 # 同时添加"蒸羊羔, 蒸熊掌"这两道菜
14 foods.extend(['蒸羊羔', '蒸熊掌'])
15 print('添加后: ', foods)

```

删除操作

```

1 '''删除操作'''
2 foods = ['烧花鸭', '烧雏鸡', '烧子鹅', '酱鸡']
3 print('删除前: ', foods)
4
5 # 删除菜"酱鸡" (使用pop方法)
6 foods.pop()
7 print('删除后: ', foods)
8
9 # 删除菜"烧雏鸡" (使用remove方法)
10 foods.remove('烧雏鸡')
11 print('删除后: ', foods)
12
13 # 删除菜"烧子鹅" (使用del方法)
14 del foods[1]
15 print('删除后: ', foods)

```

修改操作

```

1 '''修改操作'''
2 a = [1, 2, 3, 4, 5]
3 print('修改前: ', a) # 输出结果 [1, 2, 3, 4, 5]
4
5 a[1] = 6
6 print('修改后: ', a) # 输出结果 [1, 6, 3, 4, 5]

```

查询操作

```

1 '''列表的操作-查询'''
2 # in: 查询元素是否在列表中
3 # index(元素): 查询元素在列表中的索引
4 # for i in 列表名: # 遍历列表
5 #     循环体
6
7 # in的使用
8 a = [1, 2, 3, 4, 5]
9 print(3 in a)
10
11 # index()用法
12 print(a.index(3))
13
14 # 循环遍历列表
15 for i in a:
16     print(i)

```

排序

```

1  '''排序'''
2  # 列表名.sort(): 升序排序, 原列表改变
3  # 列表名.sort(reverse=True): 降序排序
4  # 列表名.reverse(): 反转列表
5  # 变量 = sorted(列表名) # 返回排序后的列表, 原列表不变
6
7  a = [1,3,2,5,4]
8  print("排序前: ",a)
9
10 # 升序排序
11 a.sort()
12 print("升序排序后: ",a)
13
14 # 降序排序
15 a.sort(reverse=True)
16 print("降序排序后: ",a)
17
18 b = [1,5,3,4,2]
19 print('排序前: ',b)
20
21 c = sorted(b)
22 print('排序后: ',c)
23
24 # 降序排序
25 d = sorted(b,reverse=True)
26 print('降序排序后: ',d)

```

3.3 字典

定义:

```

1  字典用 {} 定义
2  字典使用 键值对 存储数据, 键值对之间使用 , 分隔
3      键 key 是索引, 值 value 是数据
4      键 和 值 之间使用 : 分隔
5      值 可以取任何数据类型, 但 键 只 能使用 字符串、数字或 元组
6      键必须是唯一的

```

示例代码:

```

1  """
2  字典定义格式:
3      字典变量 = {k1:v1, k2:v2,.....}
4
5  取出元素的值:
6      字典变量[键值]
7  """
8  # 字典的定义
9  info = {'name': 'mike', 'age': 34, 'city': 'sz'}
10 print(info) # {'name': 'mike', 'age': 34, 'city': 'sz'}
11
12 # 取出元素的值: 字典变量[键值]
13 print(info['city'])

```

字典常用操作

分类	关键字 / 函数 / 方法	说明
增加	字典[键] = 值	键不存在, 会添加键值对
删除	字典.pop(键)	删除指定键值对,返回被删除的值,如果键不存在,会报错
	del 字典名[键值]	删除某个元素
	del 字典名	删除字典
	字典名.clear()	清空字典
修改	字典[键] = 值	键存在, 会修改键值对的值
查询	字典[键]	根据键取值, 键值对不存在会报错
	字典.get(键)	根据键取值, 键值对不存在返回None, 不会报错
遍历字典	for key, value in 字典.items()	遍历字典, 获取所有的键值对 (键, 值)
	字典名.keys()	查找所有的键
	字典名.values()	查找所有的值
	字典名.items()	查找所有的键和值

增加/修改操作

```
1 c = {'小明': '1234', '小红': '5678'}
2 # 取出 1234
3 print(c['小明'])
4 # 修改 1234 为 1111
5 c['小明'] = '1111'
6 print(c)
7 # 添加元素: 小黑的手机号 2222
8 c['小黑'] = '2222'
9 print(c)
```

删除操作

```
1 # 字典的变化-删
2 # 删除字典的3种方式:
3 # del 字典名[键值]    删除某个元素
4 # del 字典名          删除字典
5 # 字典名.clear()      清空字典
6
7 c = {'小明': '1234', '小红': '5678'}
8 print('删除前:', c)
9 # 删除键为小红的元素
10 del c['小红']
11 print('删除后:', c)
12 # 清空字典
13 c.clear()
14 print('清空后:', c)
```

```
15 # 删除字典
16 del c
17 # print(c) # 这句会报错，因为字典已经被删除了，无法识别当前变量
```

遍历字典

```
1 '''字典的变化-查'''
2 # 遍历字典的3种方式:
3 # keys(): 查找所有的键
4 # values(): 查找所有的值
5 # items(): 查找所有的键值对
6 c = {'小明': '1234', '小红': '5678'}
7
8 # 查找所有的键
9 for i in c.keys():
10     print(i)
11
12 # 查找所有的值
13 for i in c.values():
14     print(i)
15
16 # 查找所有的键值对
17 for i in c.items():
18     print(i, type(i))
```

3.4 字符串

定义

```
1 在 Python 中可以使用 一对双引号 " 或者 一对单引号 ' 定义一个字符串 或者 一对三引号 '''
   定义一个字符串
2
3 可以使用 索引 获取一个字符串中 指定位置的字符，索引计数从 0 开始
4
5 也可以使用 for 循环遍历 字符串中每一个字符
```

示例代码：

```
1 """
2 字符串变量 = '字符串内容'
3 说明：可以是单引号、双引号、三引号
4
5 """
6 my_str = 'python'
7 #      012345
8
9 # 取某个元素，和列表一样
10 print(my_str[0])
11
12 # 遍历取所有元素
13 for c in my_str:
14     print(c)
```

字符串的常用操作

分类	关键字 / 函数 / 方法	说明
查找	字符串.find(目标字符串, 开始索引, 结束索引)	在指定范围内, 查询目标字符串的索引, 不存在返回-1
替换	字符串.replace(原内容, 新内容, 替换次数)	返回一个替换了原内容的新字符串, 可以指定替换次数
分割	字符串.split(分割符)	以分割符拆分字符串, 返回列表
拼接		
	字符串.join(字符串列表)	以字符串来连接字符串列表中每个元素, 合并为一个新的字符串

查找操作

```
1 '''查找'''
2 # 字符串.find(目标字符串, 开始索引, 结束索引)
3 # 在指定范围内, 查询目标字符串的索引, 不存在返回-1
4
5 a = "hello python"
6
7 # 在索引1到5之间查找 h 字母的索引
8 b = a.find('h', 1, 5)
9 print("h的索引为: ",b) # 返回-1, 说明不存在
```

替换操作

```
1 '''替换'''
2 # 字符串.replace(原内容, 新内容, 替换次数)
3 # 返回一个替换了原内容的新字符串, 可以指定替换次数
4
5 a = "hello python"
6 # 将 o 替换为 3, 替换1次
7 b = a.replace('o', '3', 1)
8 print("替换后的字符串为",b)
```

分割操作

```
1 '''分割操作'''
2 # 字符串.split(分割符)
3 # 以分割符拆分字符串, 返回列表
4
5 a = "hello python"
6 # 以空格为分割符, 拆分字符串
7 b = a.split(' ')
8 print("拆分后的列表为: ",b) # 输出列表形式 ['hello', 'python']
```

拼接操作

```

1  '''拼接操作'''
2  # 字符串 + 字符串：拼接两个字符串
3  # 字符串.join(字符串列表)：以字符串来连接字符串列表中每个元素，合并为一个新的字符串
4
5  a = 'hello'
6  b = 'python'
7  # 拼接两个字符串a和b
8  c = a + b
9  print('拼接后的字符串为：',c)
10
11 d = ['world','c++']
12 print('拼接后的字符串为：', ''.join(d)) # 引号里面是两个列表元素分割时的内容
13 # 拼接后的字符串为： worldc++

```

3.5 集合

定义

- 1 集合用 {} 定义，元素 之间使用 , 分隔
- 2 可通过集合对列表中的元素完成去重功能
- 3 集合是没有顺序的，所以不能使用索引取值

集合是**无序的**，不能使用索引取值

示例代码：

```

1  # 集合：元素不会重复，{元素1，元素2.....}
2  # 定义集合
3  my_set = {1, 2, 1, 2, 1, 2}
4  print(my_set) # {1, 2}
5  print(type(my_set)) # <class 'set'>
6
7  # 通过集合完成对列表去重功能
8  name_list = ['mike', 'yoyo', 'mike']
9  # set(name_list)：给列表转换为set类型，即可完成去重功能
10 temp_set = set(name_list)
11 print(temp_set) # {'mike', 'yoyo'}

```

3.6 list、tuple、set之间类型转换

函数	说明
list(x)	将 x 转换为列表类型
tuple(x)	将 x 转换为元组类型
set(x)	将 x 转换为集合类型

示例代码：

```

1  # 列表转元组、集合 类型

```

```
2 my_list = [1, 2, 3, 5, 3, 5]
3
4 # 列表转换为元组类型
5 my_tuple = tuple(my_list)
6 print(my_tuple) # (1, 2, 3, 5, 3, 5)
7
8 # 列表转换为集合类型
9 my_set = set(my_list)
10 print(my_set) # {1, 2, 3, 5}
11
12 print('=====华丽分割线=====')
13
14 # 元组转列表、集合 类型
15 my_tuple = (1, 2, 3, 5, 3, 5)
16
17 # 元组转换为列表 类型
18 my_list = list(my_tuple)
19 print(my_list) # [1, 2, 3, 5, 3, 5]
20
21 # 元组转换为集合 类型
22 my_set = set(my_tuple)
23 print(my_set) # {1, 2, 3, 5}
24
25 print('=====华丽分割线=====')
26
27 # 集合转元组、列表 类型
28 my_set = {1, 2, 3}
29
30 # 集合转换为列表 类型
31 my_list = list(my_set)
32 print(my_list) # [1, 2, 3]
33
34 # 集合转换为元组 类型
35 my_tuple = tuple(my_set)
36 print(my_tuple) # (1, 2, 3)
```

4.函数

4.1 函数的使用

- 所谓**函数**，就是把 **具有独立功能的代码块** 组织为一个整体，在需要的时候 **调用**
- 函数的使用包含两个步骤：
 1. 定义函数 —— 在函数中编写代码，实现功能
 2. 调用函数 —— 执行编写的代码
- 函数的作用：
 - 提高代码编写效率, 代码**重用**
 - 写一次函数代码块，后面使用的时候，只需要调用函数即可，无需重新编写函数代码块

4.1.1 函数的定义

定义函数的格式如下：

```
1 def 函数名():  
2     函数内部的代码块  
3     .....
```

1. `def` 是英文 `define` 的缩写
2. 函数名称的命名应该符合标识符的命名规则
 - 可以由 **字母**、**下划线** 和 **数字** 组成
 - **不能以数字开头**
 - **不能与关键字重名**
 - 建议不要和类型同名

4.1.2 函数的调用

- 函数只是定义，不调用不会自动执行，用户看不到没有效果
- 函数调用的格式：

```
1 函数名字()
```

示例代码：

```
1  """  
2  步骤流程：  
3  函数定义：  
4  1. def 函数名字():  
5      2. 函数体，打印信息  
6  
7  函数调用：函数名字()  
8  """  
9  
10  
11 # 函数定义  
12 def say_hello():  
13     print("hello 1")  
14     print("hello 2")
```



```

15     print("hello 3")
16
17
18     print('函数调用前')
19
20     say_hello()    # 函数调用
21
22     print('函数调用后')

```

4.2 无参函数

定义函数时括号内没有参数的函数

定义:

```

1  def 函数名():
2      函数体代码

```

示例代码:

```

1  def hello():    #定义一个hello函数
2      print("hello")
3      print("python")
4  hello()    #调用hello函数

```

4.3 有参函数

- 函数定义时的参数叫 **形参**，函数调用时的参数叫 **实参**
- 带参数函数的定义格式:

```

1  def 函数名字(形参1, 形参2, .....):
2      函数体代码块

```

- 带参数函数的调用格式:

```

1  函数名字(实参1, 实参2, ..... )

```

示例代码:

```

1  # 带参数函数的函数定义
2  def func_sum(num1, num2):
3      """两个数字的求和"""
4      result = num1 + num2
5      print("%d + %d = %d" % (num1, num2, result))
6
7
8  # 带参数函数的调用
9  func_sum(20, 30)

```

4.3.1 参数不定

方法1：使用列表、元组、字典等操作

方法2：包裹位置传递

用法：

```
1 # 通过在参数名字前面加*，标识传递参数使用包裹位置
2
3 # 在调用函数时传递过来的所有参数都会被参数名字收集，根据位置合并成一个元组
```

在调用函数时传递过来的所有参数都会被参数名字收集，根据位置合并成一个**元组**

示例代码：

```
1 def add(*nums): #创建add函数
2     sum = 0 # 用sum记录数字相加的和
3     print(type(nums))
4     for num in nums: # 将 nums 列表中的所有数字遍历一次
5         sum += num # 完成nums里面的数字相加
6     return sum # 将得到的最终结果返回
7
8 # 将 [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]列表作为参数传递给add函数
9 total = add(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
10 print("1+2+3+4+5+6+7+8+9+10 = %d"%total)
```

包裹关键字传递

```
1 # 创建函数时在参数名前面加**。
2 # 通过**标识使用的是包裹关键字传递。调用时，将所有的参数名和数值组合成字典
```

示例代码：

```
1 def introduce(name, age, **more):
2     print("大家好，我的名字是：" + name)
3     print("年龄是：" + age)
4     print(more)
5 introduce("奇奇果", "12", 爱好="编程", 家乡="魔法星球")
6 introduce("美美果", "8", 爱好="学习", 生日="1月3日")
```

4.4 返回值

```
1 # 使用return关键字，将函数的计算结果返回到主程序中
```

示例代码：

```
1 def mul(num1, num2):
2     res = num1 * num2
3     return res
4 result = mul(4, 5)
5 print("乘法的结果为: ")
6 print(result)
```

4.5 匿名函数/lambda表达式

格式:

```
1 # lambda 参数: 函数执行语句块
```

注意: lambda返回的是一个函数

示例代码:

```
1 add = lambda a,b: a+b
2 print(add(3, 5)) #输出结果为3+5=8
```

5.文件

5.1 文件打开与关闭

5.1.1 文件打开函数open()

在Python, 使用open函数, 可以打开一个已经存在的文件, 或者创建一个新文件:

- 1 文件变量 = open(文件名字, 访问模式)
 - 注意: 文件名字, 访问模式都是字符串格式

示例如下:

```
1 # 'w': 只写方式打开文件, 文件不存在创建, 文件存在, 清空文件内容
2 f = open('xxx.txt', 'w')
```

说明:

访问模式	说明
r	只用于读取, 默认模式。文件不存在,会报错。
w	只用于写入。文件存在则先清空内容, 文件不存在, 创建新文件。
a	只用于写入。文件存在则在后面添加, 文件不存在, 创建新文件。

5.1.2 文件关闭函数close()

手动关闭文件

- 关闭文件, 为了释放资源
- 语法格式: 文件变量.close()

示例如下:

```
1 # 'w': 只写方式打开文件, 文件不存在创建, 文件存在, 清空文件内容
2 f = open('xxx.txt', 'w')
3
4 # 关闭文件, 为了释放资源
5 # 文件变量.close()
6 f.close()
```

自动关闭文件

```
1 with open("123.txt", "w") as f:
2     pass # 执行完缩进代码, 会自动关闭
```

5.2 文件读写函数

5.2.1 读数据

read()函数

- 格式语法:

```
1 内容变量 = 文件变量.read(n)
```

- n 为读取的字符数，不设置则全部读取

案例演练:

- xxx.txt 文件内容如下:

```
1 hello abc hello python
```

- 示例代码:

```
1 # 1. 打开文件，只读方式打开，'r'
2 # 'r': 打开文件，必须存在，不存在，报错崩溃
3 f = open('xxx.txt', 'r')
4
5 # 2. 读取文件内容
6 # 格式: 内容变量 = 文件变量.read(读取的长度)
7 # 如果read的长度不指定，默认读取全部
8
9 ret = f.read(4) # 读取4个字符
10 print(ret)
11
12 # 下次，继续接着上一次的位置
13 ret = f.read(7) # 接着读取7个字符
14 print(ret)
15
16 ret = f.read() # 接着读取剩下所有个字符
17 print(ret)
18
19 # 3. 关闭文件
20 f.close()
```

readline()函数

- readline() 每次读取一行数据
- 语法格式: 内容变量 = 文件变量.readline()

案例演练:

- xxx.txt 文件内容如下:

```
1 hello abc
2 hello python
3 hello mike
```

- 示例代码:

```
1 # 1. 打开文件，只读方式打开，'r'
```

```

2  # 'r': 打开文件，必须存在，不存在，报错崩溃
3  f = open('xxx.txt', 'r')
4
5  # 2. 读取文件内容
6  # readlines: 读取所有的行
7  # readline: 一次读取一行
8  # readline格式: 内容变量 = 文件变量.readline()
9  content = f.readline()
10 print(content)
11
12 content = f.readline()
13 print(content)
14
15 content = f.readline()
16 print(content)
17
18 # 3. 关闭文件
19 f.close()

```

readlines()函数

- `readlines()` 一次全部读出，读取所有的行，按行作为分隔条件，返回列表 (每行内容是一个元素)
- 语法格式: 内容列表变量 = 文件变量.`readlines()`

案例演练:

- `xxx.txt` 文件内容如下:

```

1 | hello abc
2 | hello python
3 | hello mike

```

- 示例代码:

```

1  # 1. 打开文件，只读方式打开，'r'
2  # 'r': 打开文件，必须存在，不存在，报错崩溃
3  f = open('xxx.txt', 'r')
4
5  # 2. 读取文件内容
6  # readlines: 读取所有的行，按行作为分隔条件
7  # 格式: 内容列表变量 = 文件变量.readlines()
8  content_list = f.readlines()
9  # '\n'换行符，代表换行
10 print(content_list)
11
12 # 通过for取出列表的所有元素
13 for v in content_list:
14     print(v)
15
16 # 3. 关闭文件
17 f.close()

```

5.2.2. 写数据

write()函数

- 使用 write() 可以完成向文件写入数据
- 语法格式：文件变量.write(所需的内容)

示例代码：

```
1  # 1. 打开文件，只写方式打开
2  f = open('xxx.txt', 'w')
3
4
5  # 2. 写文件
6  # 写文件格式：文件变量.write(所需的内容)
7  f.write('hello abc')
8  f.write(' hello python')
9
10 # 3. 关闭文件
11 f.close()
```

6.turtle库

6.1 turtle库的引入

有三种方式引入turtle库

```
1 import turtle
2 import turtle as t
3 from turtle import *
```

6.2 画笔运动命令

命令	说明
forward(距离) 可简写为: fd(距离)	向当前画笔方向移动距离
backward(距离) 可简写为bk(距离)	向当前画笔相反方向移动距离
right(角度)	右转多少度
left(角度)	左转多少度
penup()	提起笔移动, 不绘制图形, 用于另起一个地方绘制
goto(x,y)	将画笔移动到坐标为x,y的位置
pendown()	移动时绘制图形, 缺省时也为绘制
circle(半径)	画圆, 半径为正(负), 表示圆心在画笔的左边(右边)画圆
setx()	将当前x轴移动到指定位置
sety()	将当前y轴移动到指定位置
setheading(angle)	设置当前朝向为angle角度
home()	设置当前画笔位置为原点, 朝向东
dot()	绘制一个指定直径和颜色的圆点

6.3 画笔控制命令

命令	说明
fillcolor(colorstring)	绘制图形的填充颜色
color(color1, color2)	同时设置pencolor=color1, fillcolor=color2
filling()	返回当前是否在填充状态
begin_fill()	准备开始填充图形
end_fill()	填充完成
hideturtle() 可简写为: ht()	隐藏画笔的turtle形状
showturtle() 可简写为: st()	显示画笔的turtle形状

6.4 全局控制命令

命令	说明
clear()	清空turtle窗口, 但是turtle的位置和状态不会改变
reset()	清空窗口, 重置turtle状态为起始状态
undo()	撤销上一个turtle动作
isvisible()	返回当前turtle是否可见

示例代码:

```
1 # 画正方形代码如下
2 from turtle import *
3 shape('turtle') #画笔形状为小乌龟
4 for i in range(4):
5     fd(200)      # 每次走200
6     right(90)    # 右转90度
7 ht()           #隐藏画笔
8 done()         #窗口持续展示
```

7.Tkinter库的使用

7.1 基础概念

tk库的引入：

三种方法：

```
1 import tkinter
2 import tkinter as tk
3 from tkinter import *
```

- **目标：**了解 Tkinter 的基本结构，创建一个简单的窗口。

- **代码示例：**

Python复制

```
1 import tkinter as tk
2
3 # 创建主窗口
4 root = tk.Tk()
5 root.title("Tkinter 基础窗口")
6 root.geometry("300x200") # 设置窗口大小
7
8 # 显示窗口
9 root.mainloop()
```

- **功能说明：**

- `tk.Tk()`：创建主窗口。
- `title()`：设置窗口标题。
- `geometry()`：设置窗口大小。
- `mainloop()`：启动事件循环，显示窗口。

7.2 常用的tk组件

7.2.1 添加标签 (Label)

标签相关用法：

```
1 # 使用：变量=Label(窗口对象, 参数)
2 # 参数：
3     # text='字符串'：设置标签中的文本内容
4     # font=('字体', 大小)：设置字体和大小
5     # fg='颜色'：设置字体颜色
6     # bg='颜色'：设置背景颜色
7     # width=整数：设置标签的宽度
8     # height=整数：设置标签的高度
```

- **代码示例：**

Python复制

```

1  import tkinter as tk
2
3  root = tk.Tk()
4  root.title("标签示例")
5  root.geometry("300x200")
6
7  # 创建一个标签
8  label = tk.Label(root, text="欢迎使用 Tkinter! ", font=("Arial", 16),
9                  fg="blue", bg="yellow")
10 label.pack() # 使用 pack 布局方式
11 root.mainloop()

```

- 功能说明:

- `Label`: 用于显示文本或图片。
- `font`: 设置字体样式和大小。
- `fg` 和 `bg`: 分别设置文本颜色和背景颜色。
- `pack()`: 将组件添加到窗口中。

7.2.2 添加按钮 (Button)

button按钮相关用法:

```

1  # 使用: 变量=Button(窗口对象, 参数)
2  # 参数:
3      # text='字符串': 设置按钮上的文本内容
4      # command=函数名: 设置按钮被点击时执行的函数
5      # width=整数: 设置按钮的宽度
6      # height=整数: 设置按钮的高度

```

- 代码示例:

Python复制

```

1  import tkinter as tk
2
3  def on_click():
4      print("按钮被点击了!")
5
6  root = tk.Tk()
7  root.title("按钮示例")
8  root.geometry("300x200")
9
10 # 创建一个按钮
11 button = tk.Button(root, text="点击我", command=on_click, font=("Arial",
12                        14), bg="green", fg="white")
13 button.pack()
14 root.mainloop()

```

- 功能说明:

- `Button`: 用于创建按钮。
- `command`: 绑定按钮点击事件。

- 学习要点：
 - 学会创建和配置 `Button` 组件。
 - 了解如何绑定事件处理函数。

7.2.3 输入框 (Entry)

Entry相关用法：

```
1 # 使用：变量=Entry(窗口对象，参数)
2 # 参数：
3     # textvariable=变量：设置文本框中的内容，变量必须是一个StringVar类型的变量
4     # show='字符'：设置文本框中输入的字符显示为该字符，例如密码框中可以设置为'*'
5     # width=整数：设置文本框的宽度
6     # font=('字体',大小)：设置字体和大小
7     # fg='颜色'：设置字体颜色
8     # bg='颜色'：设置背景颜色
```

- 代码示例：

Python复制

```
1 import tkinter as tk
2
3 def show_text():
4     user_input = entry.get()
5     print("输入的内容是：", user_input)
6
7 root = tk.Tk()
8 root.title("输入框示例")
9 root.geometry("300x200")
10
11 # 创建一个输入框
12 entry = tk.Entry(root, font=("Arial", 14))
13 entry.pack()
14
15 # 创建一个按钮，点击后显示输入框的内容
16 button = tk.Button(root, text="显示输入", command=show_text, font=
17 ("Arial", 14))
18 button.pack()
19 root.mainloop()
```

- 功能说明：
 - `Entry`：用于用户输入文本。
 - `get()`：获取输入框的内容。

7.2.4 单选按钮 (Radiobutton)

Radiobutton相关用法：

```

1 # 使用: 变量=Radiobutton(窗口对象, 参数)
2 # 参数:
3     # text='字符串': 设置单选按钮上的文本内容
4     # variable=变量: 设置单选按钮的值, 变量必须是一个IntVar类型的变量
5     # value=整数: 设置单选按钮的值, 当单选按钮被选中时, 变量会保存该值
6     # font=('字体',大小): 设置字体和大小
7     # fg='颜色': 设置字体颜色
8     # bg='颜色': 设置背景颜色

```

- 代码示例:

Python复制

```

1 import tkinter as tk
2
3 def show_choice():
4     print("选择的是: ", var.get())
5
6 root = tk.Tk()
7 root.title("单选按钮示例")
8 root.geometry("300x200")
9
10 var = tk.StringVar() # 用于存储单选按钮的值
11
12 # 创建单选按钮
13 radio1 = tk.Radiobutton(root, text="选项1", variable=var,
14 value="Option1", command=show_choice)
15 radio1.pack()
16
17 radio2 = tk.Radiobutton(root, text="选项2", variable=var,
18 value="Option2", command=show_choice)
19 radio2.pack()
20
21 root.mainloop()

```

- 功能说明:

- Radiobutton: 用于创建单选按钮。
- variable: 绑定变量, 用于存储选择值。
- value: 设置单选按钮的值。

7.2.5 复选框 (Checkbutton)

Checkbutton相关用法:

```

1 # 使用: 变量=Checkbutton(窗口对象, 参数)
2 # 参数:
3     # text='字符串': 设置复选框中的文本内容
4     # variable=变量: 设置复选框绑定的变量
5     # onvalue=值: 设置复选框选中时的值
6     # offvalue=值: 设置复选框未选中时的值

```

- 代码示例:

Python复制

```

1  import tkinter as tk
2
3  def show_choices():
4      print("选项1:", var1.get())
5      print("选项2:", var2.get())
6
7  root = tk.Tk()
8  root.title("复选框示例")
9  root.geometry("300x200")
10
11  var1 = tk.BooleanVar() # 用于存储复选框的值
12  var2 = tk.BooleanVar()
13
14  # 创建复选框
15  check1 = tk.Checkbutton(root, text="选项1", variable=var1,
16                           command=show_choices)
17  check1.pack()
18
19  check2 = tk.Checkbutton(root, text="选项2", variable=var2,
20                           command=show_choices)
21  check2.pack()
22
23  root.mainloop()

```

- 功能说明:

- `Checkbutton`: 用于创建复选框。
- `BooleanVar`: 用于存储复选框的布尔值。

7.2.6 下拉菜单 (OptionMenu)

OptionMenu相关用法:

```

1  # 使用: 变量=OptionMenu(窗口对象, 变量, 选项列表)
2  # 参数:
3      # variable=变量: 设置下拉菜单绑定的变量
4      # 选项列表: 设置下拉菜单中的选项, 可以是一个列表或元组

```

- 代码示例:

Python复制

```

1  import tkinter as tk
2
3  def show_selection():
4      print("选择的是: ", var.get())
5
6  root = tk.Tk()
7  root.title("下拉菜单示例")
8  root.geometry("300x200")
9
10  var = tk.StringVar() # 用于存储下拉菜单的值
11  var.set("Option1") # 默认值
12
13  # 创建下拉菜单
14  options = ["Option1", "Option2", "Option3"]
15  option_menu = tk.OptionMenu(root, var, *options)

```

```

16 option_menu.pack()
17
18 # 创建按钮，点击后显示选择的内容
19 button = tk.Button(root, text="显示选择", command=show_selection)
20 button.pack()
21
22 root.mainloop()

```

- 功能说明:

- `OptionMenu`: 用于创建下拉菜单。
- `StringVar`: 用于存储下拉菜单的值。

7.3 布局管理

7.3.1 pack布局

使用最多；指定**相对位置**的布局方式；默认按照放置的先后顺序从上到下排列

pack的重要参数：**side**、**fill**、**padding**

pack中的side

相关参数:

```

1 side: 指定组件依次靠窗口排放的位置
2 可以选择的值有:
3 ·left: 靠窗口的左边排放。
4 ·right: 靠窗口的右边排放。
5 ·top: 靠窗口的顶部排放。
6 ·bottom: 靠窗口的底部排放

```

示例代码:

```

1 from tkinter import *
2 root = Tk()
3 root.title('组件排放位置')
4 root.geometry('400x300')
5
6 l1 = Label(root, text='one', bg='red')
7 l1.pack(side="left") #靠窗口左边
8
9 l2 = Label(root, text='two', bg='green')
10 l2.pack(side="right") #靠窗口右边
11
12 l3 = Label(root, text='three', bg='blue')
13 l3.pack(side="top") #靠窗口顶部
14
15 l4 = Label(root, text='four', bg='pink')
16 l4.pack(side="bottom") #靠窗口底部
17 root.mainloop() #让根窗口持续展示

```

pack中的fill

相关参数：

```
1 fill: 指定组件的填充方式
2 我们可以选择的填充方式有：
3 ·X : 组件在水平方向进行填充。
4 ·Y : 组件在垂直方向进行填充。
5 ·BOTH: 组件在水平和垂直方向都填充。
6 ·NONE: 不填充。
```

示例代码：

```
1 from tkinter import *
2 root = Tk()
3 root.title('组件排放位置')
4 root.geometry('400x300')
5
6 l1 = Label(root,text='one',bg='red')
7 l1.pack(fill=X) #靠窗口顶部
8
9 l2 = Label(root,text='two',bg='green')
10 l2.pack(fill=Y) #靠窗口顶部
11
12 l3 = Label(root,text='three',bg='blue')
13 l3.pack(fill=BOTH) #靠窗口顶部
14
15 l4 = Label(root,text='four',bg='pink')
16 l4.pack(fill=NONE) #靠窗口顶部
17 root.mainloop() #让根窗口持续展示
```

pack中的padding

相关参数：

```
1 padding: 指定组件边距，主要有内边距、外边距、水平
2 边距、垂直边距
3 padx: 水平方向的外边距。
4 pady: 竖直方向的外边距。
5 ipadx: 水平方向的内边距。
6 ipady: 竖直方向的内边距
```

示例代码：

```
1 from tkinter import *
2 root = Tk()
3 root.title('组件排放位置')
4 root.geometry('400x300')
5
6 l1 = Label(root,text='one',bg='red')
7 l1.pack(fill=X,padx=10) #靠窗口顶部
8
9 l2 = Label(root,text='two',bg='green')
10 l2.pack(fill=Y,pady=15) #靠窗口顶部
11
```



```

12 13 = Label(root, text='three', bg='blue')
13 13.pack(fill=BOTH, ipadx=20) #靠窗口顶部
14
15 14 = Label(root, text='four', bg='pink')
16 14.pack(fill=NONE, ipady=30) #靠窗口顶部
17 root.mainloop() #让根窗口持续展示

```

7.3.2 grid布局

也叫网格布局，按照**行和列**的方式来排列组件

grid把界面划分为几行几列的网格。例如我们可以把登录界面划分为一个2行2列的网格，然后将组件放置在相应的表格中

账号	输入框
密码	输入框

```

1 # 相关参数
2 #row:行 最小为0
3 #column:列 最小为0
4 #rowspan:跨行，决定组件占的行数
5 #columnspan:跨列，决定组件占的列数
6
7 #padx:水平方向的外边距
8 #pady:垂直方向的外边距
9 #ipadx:水平方向的内边距
10 #ipady:垂直方向的内边距
11
12 #N:North,北，靠父容器的上方
13 #S:South,南，靠父容器的下方
14 #E:East,东，靠父容器的右方
15 #W:West,西，靠父容器的左方
16 #NW:西北，靠父容器的左上方
17 #NE:东北，靠父容器的右上方
18 #SW:西南，靠父容器的左下方
19 #SE:东南，靠父容器的右下方

```

示例代码：

```

1 import tkinter as tk
2
3 root = tk.Tk()
4 root.title("网格布局示例")
5 root.geometry("300x200")
6
7 # 创建组件
8 label1 = tk.Label(root, text="用户名: ")
9 label1.grid(row=0, column=0)
10
11 label2 = tk.Label(root, text="密码: ")
12 label2.grid(row=1, column=0)
13
14 entry1 = tk.Entry(root)
15 entry1.grid(row=0, column=1)
16

```

```

17 entry2 = tk.Entry(root, show="*")
18 entry2.grid(row=1, column=1)
19
20 button = tk.Button(root, text="登录")
21 button.grid(row=2, column=0, columnspan=2)
22
23 root.mainloop()

```

- 功能说明：
 - `grid()`：基于网格布局，通过行和列定位组件。

7.3.3 place布局

绝对布局，通过指定组件的绝对位置来放置组件

```

1  # 相关参数
2  #x:组件的横坐标
3  #y:组件的纵坐标
4  #width:组件的宽度
5  #height:组件的高度
6
7  #relx:组件相对于父容器的横坐标，取值范围为0-1
8  #rely:组件相对于父容器的纵坐标，取值范围为0-1
9  #relwidth:组件相对于父容器的宽度，取值范围为0-1
10 #relheight:组件相对于父容器的高度，取值范围为0-1

```

示例代码：

```

1  from tkinter import *
2  root = Tk()
3  root.geometry('300x400')
4  label_1 = Label(root, text='绝对位置', bg='green') #创建Label组件的对象
5  label_1.place(x=150, y=180) #将组件放置在(150,180)的位置
6
7  label_2 = Label(root, text='相对位置', bg='red')
8  label_2.place(relx=0.5, rely=0.5) #将组件放置在父容器的中心位置
9
10 #创建label组件的对象
11 label_3 = Label(root, text='相对宽度和高度位置', bg='pink')
12 label_3.place(relwidth=0.5, relheight=0.5) #将组件放置在父容器的左上角，宽度为父容器的50%，高度为父容器的50%
13 root.mainloop()

```

7.4 事件绑定

- 目标：学习如何为组件绑定事件。
- 代码示例：

Python复制

```

1  import tkinter as tk
2
3  def on_click(event):

```

```

4     print("鼠标点击了!")
5
6     # 创建主窗口
7     root = tk.Tk() #创建一个主窗口实例
8     root.title("鼠标点击示例") #设置窗口的标题
9     root.geometry("300x200") # 设置窗口大小
10
11    # 创建一个按钮
12    button = tk.Button(root, text="点击我") #创建一个按钮组件
13    button.pack(pady=20) # 使用 pack 布局, 并在按钮周围添加一些垂直间距
14
15    # 将点击事件绑定到按钮
16    button.bind("<Button-1>", on_click) # <Button-1> 表示鼠标左键点击
17
18    # 启动事件循环
19    root.mainloop()

```

运行结果:

- 当你运行这段代码时, 会弹出一个窗口, 窗口中有一个按钮, 按钮上显示 "点击我"。
- 当你点击这个按钮时, 控制台会输出:

```
1 鼠标点击了!
```

其他事件绑定

如果你想要绑定其他类型的事件, 可以使用以下事件类型:

- `<Button-1>`: 鼠标左键点击。
- `<Button-2>`: 鼠标中键点击。
- `<Button-3>`: 鼠标右键点击。
- `<Enter>`: 鼠标进入组件区域。
- `<Leave>`: 鼠标离开组件区域。
- `<Motion>`: 鼠标移动事件。
- `<Key>`: 键盘按键事件。

8.面向对象

8.1 概念

把构成问题事务分解成各个**对象**，建立对象的目的是为了完成一个步骤，而是为了描述某个事物在整个解决问题步骤中的行为。

对于**面向对象**的思想：当需要实现一个功能的时候，看重的并不是过程和步骤，而是关心的是对象，对象能做啥

- 以吃饭 举例：找 饭馆 对象，饭馆提供菜和饭，不关心做菜和煮饭的内部过程
- 面向对象编程适合**开发大型项目**

8.2 类和对象

类：描述 **具有共同特征的事物的 抽象**，称为 **类** (class)。

类包含两个组成部分：

- **属性**：比如姓名，年龄，身高，肤色等
- **方法**：比如吃饭，睡觉，飞行，歌唱等

对象：是 **类** 的实例，是具体的实体

类和对象的关系：

- 类是对象的模板(不占内存空间)，对象是类的实例(占内存空间)。
 - 类相当于图纸，对象相当于根据图纸制造的实物。
- 每个对象必须有一个对应的类

8.2.1 类的定义

定义类格式：

```
1  #class 类名：
2  #class 类名(): # 前2个旧式写法(经典类)，不推荐
3  class 类名(object):
4      方法列表（不是真的是列表，只是多个函数的定义）
```

说明：

- 定义类时有 2 种形式：经典类和新式类，前两行注释部分则为经典类，推荐写新式类
- object 是Python 里所有类的最顶级父类
- 类名 的命名规则按照 **驼峰命名法**

定义方法(方法也叫实例方法)格式为：

```
1  class 类名(object):
2      def 方法名(self):
3          ...
```

- 方法的格式和函数类似，也可以设置参数和返回值，但是 **需要设置第一个参数为 self**
- 编辑器一般会自动生成 self ，后面会讲解 self 的作用（参考 8.2.4 self的作用）

示例代码：

```
1 class Dog(object):
2     # self自动添加的，后面有讲解
3     # 定义在类里面的函数，叫方法
4     def eat(self):
5         print('吃骨头')
6
7     def drink(self):
8         print('喝水')
```

8.2.2 创建对象和调用方法

(1) 创建对象格式：

```
1 对象变量名 = 类名()
```

示例代码：

```
1 # 定义类型
2 class Dog(object):
3     # 定义在类里面的函数，叫方法
4     def eat(self):
5         print('吃骨头')
6
7     def drink(self):
8         print('喝水')
9
10 # 创建对象格式：实例对象变量名 = 类名()
11 dog1 = Dog()
```

(2) 调用方法格式

```
1 对象变量名.方法名()
```

- 注意：虽然定义方法时设置第一个参数 `self`，但是调用方法时不要传递对应 `self` 的参数，解释器自动处理

示例代码：

```
1 class Dog(object):
2     # 定义在类里面的函数，叫方法
3     def eat(self):
4         print('吃骨头')
5
6     def drink(self):
7         print('喝水')
8
9
10 # 创建对象格式：实例对象变量名 = 类名()
11 dog1 = Dog()
12
13 # 对象变量.方法名字(), self不用处理
```

```
14 dog1.eat()
15 dog1.drink()
```

(3) 创建多个对象

```
1 对象1 = 类名()
2 对象2 = 类名()
```

示例代码：

```
1 class Dog(object):
2     # 定义在类里面的函数，叫方法
3     def eat(self):
4         print('吃骨头')
5
6     def drink(self):
7         print('喝水')
8
9     # 对象1 = 类名()
10    # 对象2 = 类名()
11
12    # 对象1
13    dog1 = Dog()
14    dog1.eat()
15    dog1.drink()
16
17    # 对象2
18    dog2 = Dog()
19    dog2.eat()
20    dog2.drink()
```

8.2.3 添加和使用属性

定义/添加属性格式：

```
1 对象变量名.属性名 = 数据
```

- 属性和变量类似，**首次赋值时会定义属性**，再次赋值改变属性

示例代码：

```
1 class Dog(object):
2     # 定义在类里面的函数，叫方法
3     def eat(self):
4         print('吃骨头')
5
6     def drink(self):
7         print('喝水')
8
9
10    # 1. 创建对象变量
11    dog1 = Dog()
12
13    # 2. 对象变量.属性 = 数值
14    dog1.age = 3
```

```

15
16 # 打印属性
17 print(dog1.age)
18
19 # 修改属性
20 dog1.age = 2
21 print(dog1.age)

```

8.2.4 self的作用

- 在Python类中规定，实现方法的第一个参数是**实例对象本身**，并且约定俗成，把其名字写为**self**。
- 某个对象调用其方法时，Python解释器会自动把这个对象作为第一个参数传递给方法
 - 通俗理解：哪个对象调用方法，方法中self就是这个对象

示例代码：

```

1 # 定义类
2 class Dog(object):
3     def print_info(self):
4         print('测试self', id(self))
5
6
7 # 创建对象1
8 dog1 = Dog()
9
10 # 打印dog1的id
11 print("调用方法前", id(dog1))
12
13 # dog1调用print_info, print_info的self就是dog1
14 # 底层调用: print_info(dog1), 解释器自动把dog1传给方法中的self
15 dog1.print_info()
16
17 print("调用方法后", id(dog1))

```

运行结果：

```

1 调用方法前 4400762352
2 测试self 4400762352

```

self的作用

- 在方法中使用 `self`，可以获取到调用当前方法的对象，进而获取到该对象的属性和方法
- self作用：为了**区分**不同对象的属性和方法

示例代码

```

1  # 定义类
2  class Dog(object):
3      def print_info(self):
4          print(self.type)
5
6
7  # 创建对象，实例化对象
8  dog1 = Dog()
9  # 添加属性
10 dog1.type = '大黄狗'
11 # 直接调用方法
12 dog1.print_info()

```

8.3 魔法方法

- 在Python中，所有以 `__` 双下划线包起来的方法，都统称为 **Magic Method**，中文称 **魔法方法**
- 魔法方法是系统提供好的方法名字，用户需重新实现它
- 魔法方法一般情况下无需手动调用，在合适时候会自动会调用

8.3.1 `__init__()`方法

- `__init__()` 方法叫做 **对象的初始化方法**，在 **创建一个对象后会被自动调用**，不需要手动调用
- `__init__()` 方法的作用：添加属性

示例代码：

```

1  """
2  __init__方法：
3      1. 作用：添加属性
4      2. 特点：创建对象的时候，实例化对象，自动调用__init__方法
5  """
6
7
8  class Dog(object):
9      def __init__(self):
10         """初始化函数，添加属性"""
11         self.type = '大黄狗'
12         print('init方法调用了')
13
14         def print_info(self):
15             """打印实例属性"""
16             print(self.type)
17
18
19     # 1. 创建对象，实例化对象，自动调用__init__方法
20     # 2. 尽管是自动调用方法，也是dog1自动调用方法，所以，self就是dog1
21     dog1 = Dog()
22
23     # 手动调用实例方法
24     dog1.print_info()

```

`__init__()` 自定义参数

- `__init__(self)` 除了默认参数 `self`，还可以设置任意个数的自定义参数，例如：
`__init__(self, x, y, z)`
- `init` 方法 **设置的自定义参数必须和创建对象时传递的参数保持一致**，例如：`对象变量名 = 类名(x, y, z)`
- 开发者可以 **设置自定义参数**，为对象的默认属性提供 **不同的初始值**

示例代码：

```

1  class Dog(object):
2      def __init__(self, _type):
3          """初始化函数，添加属性"""
4          self.type = _type
5          print('init方法调用了')
6
7      def print_info(self):
8          """打印实例属性"""
9          print(self.type)
10
11
12  # 创建对象，实例化对象，自动调用__init__方法
13  # '旺财'传递给 _type 形参
14  dog1 = Dog('旺财')
15  # 直接调用方法
16  dog1.print_info()
17
18  print('='*30)
19  dog2 = Dog('来福')
20  dog2.print_info()

```

8.3.2 `__str__()`方法

- 如果直接 `print` 打印对象，会看到创建出来的对象在内存中的地址
- 当使用 `print(对象变量名)` 输出对象的时候，只要类中定义了 `__str__()` 方法，就会打印
`__str__()` 方法返回值
- `__str__()` 方法作用主要返回对象属性信息，`print(对象变量名)` 输出对象时直接输出
`__str__()` 方法返回的描述信息
- `__str__()` 方法的返回值必须是 **字符串类型**

示例代码：

```

1  """
2  __str__方法：
3      1. 返回值必须是字符串类型
4      2. print(对象变量名) 对象变量名的位置替换为__str__()方法返回值的内容
5  """
6
7
8  class Dog(object):
9      # 添加属性，type, age
10     def __init__(self, _type, _age):
11         self.type = _type
12         self.age = _age
13
14     # 测试有定义 __str__ 方法，和没有定义，print(对象)的区别

```

```

15     def __str__(self):
16         return "类型: %s, 年龄: %d" % (self.type, self.age)
17
18
19 # 创建对象
20 dog1 = Dog('大白狗', 3)
21
22 # 如果有__str__ 方法, dog1的位置替换为__str__()方法返回值的内容
23 print(dog1)
24 print('对象描述信息为: ', dog1)

```

运行结果:

```

1  类型: 大白狗,  年龄: 3
2  对象描述信息为:  类型: 大白狗,  年龄: 3

```

8.3.3 __del__()方法

- 在对象的生命周期结束(对象销毁)时, `__del__()` 方法会自动被调用, 做一些清理工作

```

1  class Dog(object):
2      def __del__(self):
3          print('我悄悄地离开了')
4
5
6  # 设计一个函数, 在函数内容创建对象
7  # 函数调用完毕, 里面创建的对象, 生命周期结束, 自动调用__del__方法
8  def func():
9      dog1 = Dog()
10
11
12 print('函数调用前')
13 # 调用函数
14 func()
15 print('函数调用后')

```

运行结果:

```

1  函数调用前
2  我悄悄地离开了
3  函数调用后

```

8.3.4 其他专有方法

- `__init__`: 构造函数, 在生成对象时调用
- `__del__`: 析构函数, 释放对象时使用
- `__repr__`: 打印, 转换
- `__setitem__`: 按照索引赋值
- `__getitem__`: 按照索引获取值
- `__len__`: 获得长度
- `__cmp__`: 比较运算
- `__call__`: 函数调用
- `__add__`: 加运算
- `__sub__`: 减运算

- `__mul__`: 乘运算
- `__truediv__`: 除运算
- `__mod__`: 求余运算
- `__pow__`: 乘方

8.4 继承

(1) 面向对象的特性

- 面向对象的三大特性：**封装、继承、多态**
- 面向对象的封装特性：
 - 将属性和方法放到一起封装成一个整体，然后通过实例化对象来处理
 - 对类的属性和方法增加访问权限控制

(2) 私有属性

- 如果在属性名前面加了2个下划线'`__`'，则表明该属性是私有属性，否则为公有属性
- **私有属性只能在类的内部访问**

```

1  """
2  私有属性：
3      1. __ (2个下划线)开头的属性，就是私有属性
4      2. 只能在本类的内部访问，在类的外面无法直接访问
5  """
6  class Dog(object):
7      # 添加属性
8      def __init__(self):
9          self.__baby_count = 0  # 私有属性，以__ (2个下划线)开头的属性
10         self.age = 1  # 公有属性
11
12         def print_info(self):
13             print(self.__baby_count)
14
15
16 # 类的外部
17 # 创建对象
18 dog1 = Dog()
19 # print(dog1.__baby_count) # err, 私有属性，在类的外面无法直接访问
20 print(dog1.age)
21
22 dog1.print_info()

```

(3) 私有方法

- 私有方法和私有属性类似，在方法名前面加了2个下划线'`__`'，则表明该方法是私有方法
- **私有方法只能在类内部使用**

```

1  """
2  私有方法：
3      1. __ (2个下划线)开头的方法，就是私有方法
4      2. 只能在本类的内部访问，在类的外面无法直接访问
5      3. 在类的内部调用实例方法的语法格式：self.方法名()
6  """
7

```

```

8
9 class Dog(object):
10     def __init__(self):
11         self.__baby_count = 0 # 私有属性，以__(2个下划线)开头的属性
12         self.age = 1
13
14     def print_info(self):
15         print(self.__baby_count)
16         self.__leave()
17
18     # 定义一个私有方法
19     def __leave(self):
20         print('休产假了')
21
22
23 dog1 = Dog()
24 dog1.print_info()
25 # AttributeError: 'Dog' object has no attribute '__leave'
26 # dog1.__leave() # err, 外部不能访问私有方法

```

8.4.1 继承概念

(1) 继承的作用

- 继承：子类直接具备父类的能力（属性和方法）
- 作用：解决代码重用问题，提高开发效率

(2) 继承的语法格式

继承的语法格式：

```

1 class 子类名(父类名):
2     pass

```

示例代码：

```

1 # 定义一个父类
2 class Father(object):
3     # 添加一个属性， money
4     def __init__(self):
5         self.money = 9999999
6
7     def print_info(self):
8         print(self.money)
9
10
11 # 定义一个子类，继承与Father
12 class Son(Father):
13     pass
14
15
16 # 子类创建对象
17 s = Son()
18 print(s.money) # 子类私用继承过来的属性
19 s.print_info() # 子类使用继承过来的方法

```

注意：

- 子类对象调用方法有一个就近原则
 - 如果本类能找到方法，直接调用本类的方法
 - 如果本类找不到，则调用父类继承过来的方法

8.4.2 单继承和多层继承

(1) 单继承

- 单继承：子类只继承一个父类

```
1 # 定义一个父类， Animal
2 class Animal(object):
3     def eat(self):
4         print('吃东西')
5
6
7 # 定义一个子类， 只有一个父类
8 class Dog(Animal):
9     pass
10
11
12 # 创建一个子类对象
13 dog1 = Dog()
14 dog1.eat()
```

(2) 多层继承

- 多层继承：继承关系为多层传递，如生活中的 爷爷、父亲、儿子

```
1 # 定义一个爷爷类， Animal
2 class Animal(object):
3     def eat(self):
4         print('吃东西')
5
6
7 # 定义一个父亲类
8 class Dog(Animal):
9     def drink(self):
10         print('喝东西')
11
12
13 # 定义一个儿子类
14 class SuperDog(Dog):
15     pass
16
17
18 # 创建对象
19 sd = SuperDog()
20 sd.eat()
21 sd.drink()
```

8.4.3 重新父类方法super()

(1) 子类重写父类同名方法

- 父类的方法不能满足子类的需要，可以对父类的方法重写，重写父类方法的目的是为了给他扩展功能
- 在子类中定义了一个和父类同名的方法(参数也一样)，即为对父类的方法重写
- 子类调用同名方法，默认只会调用子类的

示例代码：

```
1 # 定义一个父类， Animal
2 class Animal(object):
3     # 添加一个type属性
4     def __init__(self):
5         print('Animal类中的__init__')
6         self.type = '动物'
7
8     # 设计一个方法，打印属性
9     def print_type(self):
10        print('Animal类中的print_type = ', self.type)
11
12
13 # 定义一个子类，继承与Animal
14 class Dog(Animal):
15     # __init__和父类的同名，重写父类同名方法
16     def __init__(self):
17         print('Dog类中的__init__')
18         self.type = '可爱的小狗'
19
20     # print_type和父类的同名，重写父类同名方法
21     def print_type(self):
22         print('Dog类中的print_type = ', self.type)
23
24
25 # 定义一个子类对象
26 dog1 = Dog() # 调用子类的__init__
27 dog1.print_type() # 调用子类的print_type()
```

运行结果：

```
1 Dog类中的__init__
2 Dog类中的print_type = 可爱的小狗
```

(2) 子类调用父类同名方法

- 子类调用父类同名方法：
 - 父类名.同名方法(self, 形参1,)
 - super(子类名, self).同名方法(形参1,)
 - super().同名方法(形参1,): 是方法 2 的简写，推荐的写法

示例代码：

```
1 # 定义一个父类， Animal
2 class Animal(object):
3     # 添加一个type属性
```

```

4     def __init__(self):
5         print('Animal类中的__init__')
6         self.type = '动物'
7
8     # 设计一个方法，打印属性
9     def print_type(self):
10        print('Animal类中的print_type = ', self.type)
11
12
13 # 定义一个子类，继承与Animal
14 class Dog(Animal):
15     # __init__和父类的同名，重写父类同名方法
16     def __init__(self):
17         print('Dog类中的__init__')
18         self.type = '可爱的小狗'
19
20     # print_type和父类的同名，重写父类同名方法
21     def print_type(self):
22         print('Dog类中的print_type = ', self.type)
23         print('='*20)
24
25     # 调用父类同名函数
26     # 方法1: 父类名.同名方法(self, 形参1, ..... )
27     Animal.__init__(self)
28     Animal.print_type(self)
29     print('=' * 20)
30
31     # 方法2: super(子类名, self).同名方法(形参1, ..... )
32     super(Dog, self).__init__()
33     super(Dog, self).print_type()
34     print('=' * 20)
35
36     # 方法3: super().同名方法(形参1, ..... ) # 是 4.2 方法的简写
37     # 推荐使用的方法
38     super().__init__()
39     super().print_type()
40
41
42 # 定义一个子类对象
43 dog1 = Dog() # 调用子类的__init__
44 dog1.print_type() # 调用子类的print_type()

```

运行结果:

```

1  Dog类中的__init__
2  Dog类中的print_type =  可爱的小狗
3  =====
4  Animal类中的__init__
5  Animal类中的print_type =  动物
6  =====
7  Animal类中的__init__
8  Animal类中的print_type =  动物
9  =====
10 Animal类中的__init__
11 Animal类中的print_type =  动物

```

8.4.4 多继承

(1) 多继承

- 所谓多继承，即子类有多个父类，并且具有它们的特征。
- 多继承的语法格式：

```
1 class 子类名(父类1, 父类2, .....):  
2     pass
```

示例代码：

```
1 # 定义2个类，它们没有继承关系，是平级的  
2 class SmallDog(object):  
3     def eat(self):  
4         print('吃小东西')  
5  
6  
7 # 再定义一个类  
8 class BigDog(object):  
9     def drink(self):  
10        print('大口喝水')  
11  
12  
13 # 定义一个子类，多继承于上面2个父类  
14 class SuperDog(SmallDog, BigDog):  
15     pass  
16  
17  
18 # 定义子类对象，调用方法  
19 sd = SuperDog()  
20 sd.eat()  
21 sd.drink()
```

运行结果：

```
1 吃小东西  
2 大口喝水
```

(2) 类的继承顺序

- 查看类的继承顺序：类名. `__mro__`

示例代码：

```
1 # 定义2个类，它们没有继承关系，是平级的  
2 class SmallDog(object):  
3     def eat(self):  
4         print('吃小东西')  
5  
6  
7 # 再定义一个类  
8 class BigDog(object):
```



```

9         def drink(self):
10             print('大口喝水')
11
12
13     # 定义一个子类，多继承于上面2个父类
14     class SuperDog(SmallDog, BigDog):
15         pass
16
17
18     # 查看类的继承顺序
19     print(SuperDog.__mro__)

```

运行结果：

```

1 (<class '__main__.SuperDog'>, <class '__main__.SmallDog'>, <class
  '__main__.BigDog'>, <class 'object'>)

```

(3) 调用父类同名方法

默认调用情况

- 如果继承过来的2个父类的方法同名，默认调用先继承父类的同名方法

```

1     # 定义2个类，它们没有继承关系，是平级的
2     class SmallDog(object):
3         def eat(self):
4             print('吃小东西')
5
6
7     # 再定义一个类
8     class BigDog(object):
9         def eat(self):
10            print('啃大骨头')
11
12
13    # 定义一个子类，多继承于上面2个父类
14    class SuperDog(SmallDog, BigDog):
15        pass
16
17
18    # 定义子类对象，调用方法
19    sd = SuperDog()
20    sd.eat() # 默认先调用先继承的父类，即 SmallDog

```

运行结果：

```

1 吃小东西

```

子类调用父类同名方法

- 子类调用父类同名方法：
 1. 父类名.同名方法(self, 形参1,): 调用指定的父类

2. `super(类名, self).同名方法(形参1, .....)`：调用继承顺序中类名的下一个类的同名方法
3. `super().同名方法(形参1, .....)`：调用先继承父类的同名方法

示例代码：

```
1  # 定义2个类，它们没有继承关系，是平级的
2  class SmallDog(object):
3      def eat(self):
4          print('吃小东西')
5
6
7  # 再定义一个类
8  class BigDog(object):
9      def eat(self):
10         print('啃大骨头')
11
12
13 # 定义一个子类，多继承于上面2个父类
14 class SuperDog(SmallDog, BigDog):
15     def eat(self):
16         print('吃蟠桃')
17         print('='*20)
18
19         # 子类调用父类同名方法：
20         # 1. 父类名.同名方法(self, 形参1, ..... )
21         SmallDog.eat(self) # 调用SmallDog的eat()
22         print('=' * 20)
23
24         # 2. super(类名, self).同名方法(形参1, .....): 调用继承顺序中类名的下一个类的同名方法
25         # 继承顺序中，SmallDog的下一个类是BigDog,所以，调用BigDog的eat()
26         super(SmallDog, self).eat()
27         print('=' * 20)
28
29         # 3. super().同名方法(形参1, .....): 调用先继承父类的同名方法
30         super().eat()
31
32
33 # 定义子类对象，调用方法
34 sd = SuperDog()
35 sd.eat()
```

运行结果：

```
1  吃蟠桃
2  =====
3  吃小东西
4  =====
5  啃大骨头
6  =====
7  吃小东西
```

8.4.5 私有和继承

- 父类中的私有方法、属性不能直接继承使用
- 可以通过调用继承的父类的共有方法，间接的访问父类的私有方法、属性

```
1  # 定义一个父类， Animal
2  class Animal(object):
3      # 添加一个type属性
4      def __init__(self):
5          self.__type = '动物' # 私有
6
7      def __leave(self): # 私有
8          print('休产假3个月')
9
10     # 通过公有方法，间接访问私有元素
11     def use_private(self):
12         print(self.__type)
13         self.__leave()
14
15
16     # 定义一个子类
17     class Dog(Animal):
18         def test(self):
19             # print(self.__type) # err, 私有不能直接继承使用
20             # self.__leave() # err, 私有不能直接继承使用
21             pass
22
23
24     # 创建子类对象
25     dog1 = Dog()
26     dog1.use_private()
```

8.4.6 多态

- 多态：多种形态，调用同一个函数，不同表现
- 因为Python是动态语言，站在用户的角度，本身就是多态，不存在非多态的情况
- 实现多态的步骤：
 1. 实现继承关系
 2. 子类重写父类方法
 3. 通过对象调用该方法

示例代码：

```
1  """
2  1. 多态：多种形态，调用同一个函数，不同表现
3
4  2. 实现多态的步骤：
5      1. 实现继承关系
6      2. 子类重写父类方法
7      3. 通过对象调用该方法
8
9  """
10
11
12  # 定义一个父类， Animal
```

```

13 class Animal(object):
14     def eat(self):
15         print('吃东西')
16
17
18 # 定义一个子类Dog, 继承于Animal
19 class Dog(Animal):
20     def eat(self):
21         """重写父类方法"""
22         print('啃骨头')
23
24
25 # 定义一个子类Cat, 继承于Animal
26 class Cat(Animal):
27     def eat(self):
28         """重写父类方法"""
29         print('吃小鱼')
30
31
32 # 定义一个函数, 用于测试多态
33 def func(temp):
34     temp.eat()
35
36
37 # 创建子类对象
38 d = Dog()
39 c = Cat()
40
41 # 调用同一个函数, 不同表现
42 # 传递d参数, 调用Dog的eat()
43 # 传递c参数, 调用Cat的eat()
44 func(d) # 啃骨头
45 func(c) # 吃小鱼

```

8.5 属性和方法

8.5.1 实例属性、类属性定义

实例属性

- 通过在 `__init__` 方法里面给实例对象添加的属性
- 在类的外面, 直接通过实例对象添加的属性
- **实例属性**必须通过**实例对象**才能访问

```

1  # 定义类
2  class 类名(object):
3      def __init__(self):
4          self.实例属性变量1 = 数值1
5          self.实例属性变量2 = 数值3
6
7  # 创建实例对象
8  实例对象名 = 类名()
9
10 # 添加属性
11 实例对象名.实例属性变量3 = 数值3

```

类属性

- 类属性就是 **类对象** 所拥有的属性，它被 **该类的所有实例对象** 所共有。
- 定义在**类里面**，**类方法外面**的变量就是**类属性**
- 类属性可以使用 **类名** 或 **实例对象** 访问，**推荐使用类名访问**

```

1  # 定义类
2  class 类名(object):
3      类属性变量 = 数值1
4
5      def __init__(self):
6          pass

```

示例代码

```

1  class Dog(object):
2      # 类属性
3      count = 0
4
5      def __init__(self):
6          # 实例属性
7          self.name = '大黄狗'
8
9
10 # 类属性同构类名访问，格式：类名.类属性名字
11 print(Dog.count)
12
13 # 创建实例对象
14 dog1 = Dog()
15 print(dog1.count) # 实例对象访问类属性
16 print(dog1.name) # 实例属性

```

运行结果：

```

1  0
2  0
3  大黄狗

```

8.5.2 类属性和实例属性的区别

- 类属性就是 **类对象** 所拥有的属性，它被 **该类的所有实例对象** 所共有。
- **实例属性** 要求 **每个对象** 为其 **单独开辟一份内存空间**，只属于某个实例对象的

示例代码：

```
1  """
2  1. 定义一个类属性count，用于记录实例对象初始化的次数
3  2. __init__添加实例属性name，每初始化1次，类属性count加1
4  """
5
6  class Dog(object):
7      # 类属性
8      count = 0
9
10     def __init__(self, _name):
11         # 实例属性
12         self.name = _name
13
14         # 每初始化一次，类属性数量加1
15         Dog.count += 1
16
17
18     # 打印类属性的值
19     print(Dog.count)
20
21     # 创建1个对象
22     d1 = Dog('旺财')
23     # 打印：实例属性，类属性
24     print(d1.name, Dog.count)
25
26     d2 = Dog('旺钱')
27     print(d2.name, Dog.count)
28
29     d3 = Dog('旺仔')
30     print(d3.name, Dog.count)
31
32     # 通过实例对象，访问类属性
33     print(d1.count, d2.count, d3.count)
```

运行结果：

```
1  0
2  旺财 1
3  旺钱 2
4  旺仔 3
5  3 3 3
```

8.5.3 实例属性、类属性注意事项

(1) 修改类属性

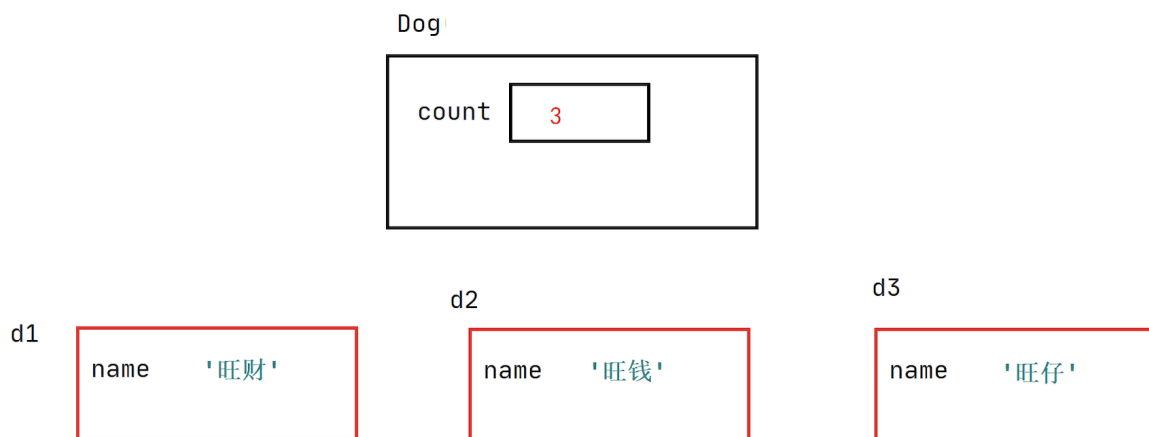
- 类属性只能通过类对象修改，不能通过实例对象修改

```
1  # 类属性修改，只能通过类名修改，不能通过对象名修改
```

```

2
3 class Dog(object):
4     # 类属性
5     count = 0
6
7     # 通过类名修改
8     Dog.count = 1
9     print(Dog.count)
10
11 print('='*30)
12 # 对象名.变量 = 数据 默认操作给实例对象添加实例属性，已经不能操作类属性
13 # 如果类属性名字和实例属性名字相同，实例对象名只能操作实例属性
14 d1 = Dog()
15 d1.count = 250
16
17 print(Dog.count, d1.count)

```



实例属性只属于某个对象，类属性属于类的，也是所有对象共有的

运行结果：

```

1 1
2 =====
3 1 250

```

(2) 类属性和实例属性同名

- 如果类属性和实例属性同名，实例对象名只能操作实例属性
- 结论：操作类属性建议使用类名，避免不必要的麻烦

```

1 class Dog(object):
2     # 类属性
3     count = 666
4
5     def __init__(self):
6         self.count = 250 # 实例属性
7
8     # 创建对象
9     # 如果类属性和实例属性同名，实例对象名只能操作实例属性
10    d1 = Dog()
11    print(Dog.count, d1.count)

```

运行结果:

```
1 | 666 250
```

(3) 私有类属性

- 类属性也可以设置为 **私有**，前边添加两个下划线 `__`

```
1 class Dog(object):
2     # 类属性
3     __count = 0
4
5 print(Dog.__count) # 类的外面，不能直接访问私有类属性，err
```

8.5.4 类方法和静态方法

(1) 类方法

- **类对象所拥有的方法**，主要为了在没有创建实例对象前提下，处理类属性
- 需要用装饰器 `@classmethod` 来标识其为类方法
- 对于类方法，**第一个参数必须是类对象(代表类)**，一般以 `cls` 作为第一个参数，这个参数不用人为传参，解释器会自动处理

```
1 """
2 类方法：为了方便处理类属性
3     1. 用装饰器 @classmethod 来标识其为类方法
4     2. 一般以 cls 作为第一个参数，代表当前这个类，这个参数不用人为传参，解释器会自动处理
5     3. 类方法调用：
6         3.1 类名.类方法()      推荐用法
7         3.2 实例对象名.类方法()
8 """
9
10
11 class Dog(object):
12     # 类属性
13     count = 0
14
15     # 定义类方法
16     @classmethod
17     def print_num(cls): # 参数cls代表当前的类
18         # print('count = ', Dog.count)
19         print('count = ', cls.count)
20
21
22 # 调用类方法
23 Dog.print_num()
```

(2) 静态方法

- 需要通过装饰器 `@staticmethod` 来进行修饰，**静态方法默认情况下，既不传递类对象也不传递实例对象（形参没有self/cls）**。
- 当方法中 **既不需要使用实例对象，也不需要类对象**时，定义静态方法
- **取消不需要的参数传递**，有利于 **减少不必要的内存占用和性能消耗**
- 静态方法也能够通过 **实例对象** 和 **类对象(类名)** 去访问。


```
1  """
2  静态方法:
3      1. 需要通过装饰器@staticmethod来进行修饰默认情况下
4      2. 既不传递类对象也不传递实例对象（形参没有self/cls）
5      3. 静态方法调用:
6          3.1 类名.静态方法()    推荐用法
7          3.2 实例对象名.静态方法()
8  """
9
10
11 class Dog(object):
12     # 定义静态方法
13     @staticmethod
14     def normal_func():
15         print('一个和实例属性、类属性没有关系的普通方法')
16
17
18 # 调用静态方法
19 Dog.normal_func()
```